YAPL-S Lexer & Parser Implementation

YAPL Extended with Native String Operations
CS 327: Compilers

Group 6
21110202 21110060 22110099 21110210

Spring 2026

Contents

1 Introduction

YAPL-S is a variant of the YAPL language (a subset of ANSI 2011 C Standard) as taught in **CS327: Compilers** course at IITGN. It retains the core syntax and semantics of YAPL (language constructs, supported data types and supported operators), but additionally introduces native support for basic string manipulation as YAPL does not support `string` preprocessors.

This specification details the extensions made to the base YAPL language to support Pythonic **F-Strings** (Format Strings) and **String Concatenation** as native language features, eliminating the need for the `<string.h>` library for common string operations.

Note: The YAPL-S and the original YAPL language specifications are provided in Appendix A and Appendix B, respectively.

2 YAPL-S: Lexer and Parser Architecture Guide

In this document, we will go through the architectural additions and internal mechanisms used to extend the base YAPL compiler into YAPL-S. It assumes familiarity with the YAPL-S language specification and focuses on how the new features were implemented in Lex (`yapl_s.l`) and Yacc (`yapl_s.y`), with a special emphasis on the **state-based tokenizer**.

2.1 Architectural Delta: YAPL vs. YAPL-S

The base YAPL language is a standard subset of C11 that lacks native string manipulation capabilities. YAPL-S introduces two major semantic features requiring deep modifications to both the lexer and the parser:

- **Binary String Concatenation (`@`)**:
  - **Lexer Addition**: A simple pass-through token for `@`.
  - **Parser Addition**: A new `concatenation_expression` tier was injected strictly between `shift_expression` and `relational_expression`. This specifically preserves standard C pointer arithmetic precedence (e.g., `ptr + 1` happens before concatenation).
- **Interpolated F-Strings (`f"..."`)**:
  - **Lexer Addition**: A multi-state, stack-based tokenization logic to handle *context-sensitive scanning.*
  - **Parser Addition**: A left-recursive grammar rule that combines text literals and C-expressions.

3 YAPL-S Lexer: Context-Sensitive Tokenization

Standard Lex is a **Finite State Automaton (FSA)**. FSAs have no memory and cannot balance nested structures.

If we parse an F-string like `f"Value:{x + 1}"`, the lexer encounters a severe conflict:

- In String (dubbed String/`FSTRING` Mode hereafter), a space is a literal character that must be kept. The word `int` is just the letters i-n-t.
- Inside the {} (dubbed Code/`INITIAL` Mode hereafter), a space is whitespace to be ignored. The word `int` is a language keyword.

Because Lex operates ahead of the Yacc parser, Yacc cannot tell Lex how to interpret the characters. The Lexer itself must maintain its own context. This led us to *Context-Sensitive Lexing.*

3.1 Python PEP 701: Syntactic formalization of f-strings

The problem of nested interpolation is not unique to YAPL-S. Python faced this exact limitation. Before Python 3.12, the CPython lexer used a "hack" where it read the entire `f"..."` as a single, massive string token, deferring the extraction of the inner {...} expressions to the parser. This meant you could not easily nest f-strings or use the same quote types inside them.

In PEP 701, Python completely overhauled its lexer to use a State Stack.

> "This PEP proposes formalizing the f-string syntax... The parser will be modified to understand f-strings natively. This is achieved by adding a new state to the tokenizer... When the tokenizer finds an f-string, it enters a specific mode... When it finds a brace { inside an f-string, it drops back to the regular mode, but keeps track of the fact that it was inside an f-string using a stack." - Python PEP 701

YAPL-S adopts this exact modern architecture, effectively bypassing the limitations of older string interpolation designs.

3.2 YAPL-S Stack-Based Lexer Implementation

To replicate the PEP 701 behavior in standard Lex, the YAPL-S lexer creates a simulated *Pushdown Automaton (PDA)* using a C-array stack.
Core Components in `yapl_s.l`:

- **State Stack**: An array `int state_stack[100]` with `push_state()` and `pop_state()` functions. This remembers the "modes" the lexer was in.
- **Added New Start Condition (`%x FSTRING`)**: An exclusive lexical state. When active, normal C code rules (like keywords or ignoring whitespace) are completely disabled. Used to scan string literals as intended.
- **Brace Counter (`brace_depth`)**: An integer that counts nested { and } while in normal C Code Mode.

How the Modes Switch:

- **Entering**: When Lex sees `f"`, it saves its current state to the stack and executes `BEGIN(FSTRING)`. Now, it tokenizes string literals as is.
- **Interpolating**: When Lex sees { inside an F-string, it pushes `FSTRING` to the stack, resets `brace_depth = 0`, and executes `BEGIN(INITIAL)`. Now, it perfectly tokenizes C code.
- **Exiting**: When Lex sees } in `INITIAL` mode, it checks `brace_depth`. If `brace_depth == 0` and the top of the stack is `FSTRING`, it knows this brace closes an interpolation, not a C Code block. It pops the stack and executes `BEGIN(FSTRING)`.

3.3 The <FSTRING> State and Token Rules

To safely parse string contents without accidentally triggering standard C keywords (like `if` or `while`), the lexer defines an exclusive start condition via `%x FSTRING`. When the lexer enters this state, it ignores all normal C rules and operates inside a strict "string sandbox."

Within this sandbox, the lexer evaluates a highly specific set of rules to break the string down into its component tokens:

- **Raw Text Consumption**: The most important regular expression of the String Mode is `([^"{}\\\n]|{ES})+`. It rapidly gobbles up continuous blocks of standard text and valid C-escape sequences (like `\n` or `\t`), stopping only when it hits a boundary character: a quote, a brace, a newline, or an invalid backslash.

- **Escaping Literal Braces**: YAPL-S mirrors modern standards (like Python) by using brace doubling. If the lexer encounters {{ or }} while inside the `<FSTRING>` state, it explicitly matches them via the `"{{"|"}}"` rule and emits them as safe `STRING_LITERAL_PART` tokens. This prevents the lexer from falling into `INITIAL` mode, allowing programmers to safely write strings like:

```
    char *set = f"Math Set: {{1, 2, {dynamic_val}}}";

```

- **Interpolation Entry**: If a single { is encountered, the sandbox pauses. The lexer pushes the `FSTRING` state to the stack, sets `brace_depth = 0`, and executes `BEGIN(INITIAL)`. This hands control back to the standard C-lexer rules, allowing the inner expression to be tokenized normally. It simultaneously emits an `INTERPOLATION_START` token for the parser.
- **String Termination**: When the lexer encounters an unescaped ¨, it knows the current string segment is complete. It pops the previous state from the stack (using `pop_state()`), executes a `BEGIN` to return to that state, and emits `FSTRING_END`.
- **Strict Error Handling**: To prevent runaway parsing, the sandbox contains strict error-catching rules:
  - **Unescaped Newlines**: F-strings, like standard C-strings, are not allowed to silently span multiple lines. Catching a raw `\n` triggers a `[lexer error]: unescaped newline in f-string`.
  - **Invalid Escapes**: A stray backslash (\) that does not form a valid escape sequence instantly halts the lexer with an `invalid escape sequence` error.

4 YAPL-S Parser

To support the new string features without breaking standard ANSI C11 behavior, the YAPL-S Yacc grammar required two major architectural injections.

4.1 The Concatenation Operator Precedence Tier

The base YAPL language groups operators into strictly ordered tiers (e.g., `multiplicative_expression`, `additive_expression`, `shift_expression`).

To safely introduce the `@` operator, a new `concatenation_expression` tier was inserted directly between `shift_expression` and `relational_expression`.

```
concatenation_expression
    : shift_expression
    | concatenation_expression '@' shift_expression
    ;
```

By making concatenation derive from shift (and ultimately additive) expressions, the parser intrinsically guarantees that C pointer arithmetic (like `str + 1`) resolves before string concatenation.

4.2 Left-Recursive String Assembly

When the lexer breaks an F-string into chunks, the parser must stitch them back together dynamically. This is achieved using a left-recursive grammar rule for `f_string_body`.

```
f_string_body
    : /* empty */
    | f_string_body STRING_LITERAL_PART
    | f_string_body interpolation_block
    ;
```

Left recursion is highly efficient for LALR parsers like Yacc. It ensures the parser never has more than two or three string chunks on its internal stack at any given time. As each new `STRING_LITERAL_PART` or `interpolation_block` arrives, the parser immediately reduces it into the growing `f_string_body` node.

Note: Refer to the YAPL-S language specifications as provided in Appendix A to view all the changes in the grammar.

5 Tracing an Example Nested F-String

Let's trace a highly complex, deeply nested edge case: `f"A␣{␣f"{b}"␣}␣{␣f"C"}"`, where ␣ denotes a space (for better readability).

This string contains an outer F-string, which contains two interpolation blocks. The first interpolation block contains another F-string, which itself interpolates a variable `b`. Furthermore, there is intentional spacing inside the code blocks which the lexer must know to ignore.

Refer to Table 1 for a step-by-step trace of how the lexer and parser collaborate to evaluate this sequence.

6 Shared Artifacts

Along with this document, we have attached the YAPL-S Lex (`yapl_s.l`) and Yacc (`yapl_s.y`) file. Using the `Makefile`, one can generate the lexers and parsers of YAPL-S and start tokenizing YAPL-S codes. Details of running the same are given in a dedicated `README.md` file.

We have also provided a test suite which contains both `positive` (cases where YAPL-S must successfully parse the code) and `negative` (cases where YAPL-S must fail parsing/lexing gracefully) examples. The outputs of the tokenizer along with some important grammar reductions are demonstrated as debug output in the corresponding `.txt` files.

The same can be found here: Github Repo

Input	Lexer Mode (Before → After)	Lexer Stack (After Push/Pop)	Token Emitted	Parser Action
f"	INITIAL → FSTRING	[INITIAL]	FSTRING_START	**Shift** to Parse Stack.
A_	FSTRING	[INITIAL]	STRING_LITERAL_PART	**Shift** then **Reduce** against empty f_string_body to start building.
{	FSTRING → INITIAL	[INITIAL, FSTRING]	INTERPOLATION_START	**Shift**. We are now parsing C code.
_	INITIAL	[INITIAL, FSTRING]	*(Ignored)*	Lexer discards whitespace in code mode.
f"	INITIAL → FSTRING	[INITIAL, FSTRING, INITIAL]	FSTRING_START	**Shift**. Starting the first inner F-string.
{	FSTRING → INITIAL	[INITIAL, FSTRING, INITIAL, FSTRING]	INTERPOLATION_START	**Shift**. Preparing to parse the inner variable.
b	INITIAL	[INITIAL, FSTRING, INITIAL, FSTRING]	IDENTIFIER	**Shift** then **Reduce** ID → primary_expr → expr.
}	INITIAL → FSTRING	[INITIAL, FSTRING, INITIAL]	INTERPOLATION_END	**Shift** then **Reduce** the inner interpolation_block.
"	FSTRING → INITIAL	[INITIAL, FSTRING]	FSTRING_END	**Shift** then **Reduce** the entire inner f_string_expression.
_	INITIAL	[INITIAL, FSTRING]	*(Ignored)*	Lexer discards whitespace again.
}	INITIAL → FSTRING	[INITIAL]	INTERPOLATION_END	**Shift** then **Reduce** the first outer interpolation_block and append it.
_	FSTRING	[INITIAL]	STRING_LITERAL_PART	**Shift** then **Reduce** to append the space between the blocks.
{	FSTRING → INITIAL	[INITIAL, FSTRING]	INTERPOLATION_START	**Shift**. Opening the second outer interpolation block.
_	INITIAL	[INITIAL, FSTRING]	*(Ignored)*	Whitespace in code mode.
f"	INITIAL → FSTRING	[INITIAL, FSTRING, INITIAL]	FSTRING_START	**Shift**. Starting the second inner F-string.
C	FSTRING	[INITIAL, FSTRING, INITIAL]	STRING_LITERAL_PART	**Shift** then **Reduce** to an inner f_string_body.
"	FSTRING → INITIAL	[INITIAL, FSTRING]	FSTRING_END	**Shift** then **Reduce** to an f_string_expression.
}	INITIAL → FSTRING	[INITIAL]	INTERPOLATION_END	**Shift** then **Reduce** the second outer interpolation_block and append.
"	FSTRING → INITIAL	[] (Empty)	FSTRING_END	**Shift**. The outer F-string is complete.

Table 1: Step-by-Step Lexer and Parser Trace for Nested F-String Evaluation

A Appendix: YAPL-S Language Specification

A.1 Lexical Conventions

The lexical analyzer distinguishes tokens based on the standard C rules, with the following additions for handling string interpolation.

A.1.1 String Concatenation Operator

The following operator is added to the language:

Operator	Symbol	Description
Concatenation	`@`	Binary operator for joining two string literals.

Table 2: New Operators in YAPL-S

A.1.2 Interpolated String Literals

An interpolated string literal (Pythonic F-String) is distinguished from a standard string literal by the prefix `f`.

Syntax:

```
f" text { expression } text "
```

The lexical analyzer would process F-Strings using the `lex` tool to emit the following distinct token types:

- `FSTRING_START`: The sequence `f"`.
- `STRING_LITERAL_PART`: A sequence of characters inside the F-String that are *not* part of an interpolation.
- `INTERPOLATION_START`: The { character inside an F-String.
- `INTERPOLATION_END`: The } character inside an F-String.
- `FSTRING_END`: The closing " character.

Note: Standard escape sequences (e.g., \n, \") are valid within STRING_LITERAL_PART.

A.2 Syntax

The grammar of YAPL-S extends the YAPL grammar.

A.2.1 Operator Precedence and Associativity

The `@` operator is inserted strictly between the **Shift** and **Relational** tiers. This ensures `char *` arithmetic (addition/subtraction) takes priority over concatenation.

Prec.	Operator Class	Operators	Associativity
1	Primary/Postfix	`() [] -> .`	L-to-R
2	Unary	`* & - !`	R-to-L
3	Multiplicative	`* / %`	L-to-R
4	Additive	`+ -`	L-to-R
5	Shift	`<< >>`	L-to-R
6	**Concatenation**	`@`	**L-to-R**
7	Relational	`< > <= >= <=>`	L-to-R
8	Equality	`== !=`	L-to-R

Table 3: Revised Operator Precedence Table

A.2.2 Grammar Specification

The grammar hierarchy reflects the precedence table.

```
// 1. Relational expressions derive from concatenation
relational_expression
  : concatenation_expression
  | relational_expression '<' concatenation_expression
  | relational_expression '>' concatenation_expression
  | relational_expression LE_OP concatenation_expression
  | relational_expression GE_OP concatenation_expression
  | relational_expression TH_OP concatenation_expression
  ;

// 2. Concatenation derives from Shift
// This ensures @ has lower precedence than <<, +, *, etc.
concatenation_expression
    : shift_expression
    | concatenation_expression '@' shift_expression
    ;

// 3. Shift derives from Additive (Standard C)
shift_expression
  : additive_expression
  | shift_expression LEFT_OP additive_expression
  | shift_expression RIGHT_OP additive_expression
  ;

// 4. Primary Expression updated for F-Strings
primary_expression
  : IDENTIFIER
  | constant
  | string
  | '(' expression ')'
  | generic_selection
    | f_string_expression   /* Extension */
  ;

// 5. F-String Structure
```

```
f_string_expression
    : FSTRING_START f_string_body FSTRING_END
    ;

f_string_body
    : /* empty */
    | f_string_body STRING_LITERAL_PART
    | f_string_body interpolation_block
    ;

interpolation_block
    : INTERPOLATION_START expression INTERPOLATION_END
    ;
```

A.3 Semantics

A.3.1 F-String Evaluation

An F-String expression evaluates to a value of type `char *`.

1. **Evaluation Order:** The expressions inside the interpolation blocks {...} are evaluated from left to right at runtime.
2. **Type Conversion:** The result of each interpolated expression is implicitly converted to its string representation.
   - `int`: Converted to decimal string (e.g., `10` → `"10"`).
   - `float/double`: Converted to standard floating-point representation.
   - `char *`: Copied as-is.
   - `char`: Converted to a single-character string.
3. **Construction:** The literal text parts and the converted expression strings are concatenated in order.
4. **Memory:** The result is stored in a newly allocated memory block (heap). The runtime environment manages this allocation.

A.3.2 The Concatenation Operator (`@`)

The `@` operator performs string concatenation.

- **Operands:** Both operands must evaluate to type `char *`.
- **Result:** A new `char *` containing the contents of the left operand followed immediately by the right operand.
- **Error Handling**: The compiler shall issue a Type Error if an operand is numeric (e.g., `str @ 5` is illegal).
- **Behavior**: Resultant string is automatically null-terminated.
  1. Compute length of left string (L_1)$and right string(L_2)$.
  1. Allocate $\mathrm{L}_1 + L_2 + 1 bytes on the heap.$
  1. Copy Left String $\rightarrow CopyRightString \rightarrow NullTerminate.$

- **Rationale for Precedence**: Given `str1 @ str2 + 1`:
  - Additive (+) runs first: `str2 + 1` advances the pointer by 1 char.
  - Concatenation (@) runs second: Joins `str1` with the substring.
  - This prevents the inefficiency of allocating a new string only to immediately skip its first character.

A.4 Examples and Use Cases

A.4.1 Initialization and Assignment

Native strings can be created dynamically without reliance on `sprintf` or `strcat`.

```
int main() {
    int id = 42;
    double score = 98.5;
    char *user = "Alice";

    // --- Standard C11 Approach (Tedious) ---
    // Requires <stdio.h>, <stdlib.h>, <string.h>
    // char buffer[100];
    // sprintf(buffer, "User: %s (ID: %d)", user, id);

    // --- YAPL-S Approach (Native) ---
    // F-Strings handle implicit memory allocation and formatting
    char *status = f"User: {user} (ID: {id})";

    // String Concatenation using the @ operator
    char *full_msg = status @ " - Status: Active";

    // full_msg is now: "User: Alice (ID: 42) - Status: Active"

    return 0;
}
```

A.4.2 Usage in Function Calls

F-Strings evaluate to `char *`, allowing them to be passed directly to any function expecting a C string.

```
void log_event(char *msg) {
    // Implementation for logging...
}

int main() {
    int x = 10, y = 20;

    // Direct usage in printf (replaces %s format specifiers)
    printf(f"Coordinates: x={x}, y={y}\n");

    // Usage in custom functions with embedded arithmetic
    // The expression {x + y} is evaluated before string construction
    log_event(f"System Check: {x + y} units detected.");

    return 0;
}
```

A.4.3 Interaction with Pointer Arithmetic

Because `@` has lower precedence than additive operators (`+`, `-`), standard C pointer arithmetic behaves predictably when mixed with concatenation.

```c
int main() {
    char *h = "Hello";
    char *w = "World";

    // Scenario A: Pointer Arithmetic on the Operand (Standard Precedence)
    // 1. ("World" + 1) advances pointer to "orld"
    // 2. "Hello" @ "orld" concatenates
    char *subset = h @ w + 1;
    // Result: "Helloorld"

    // Scenario B: Pointer Arithmetic on the Result (Grouped)
    // 1. ("Hello" @ "World") creates "HelloWorld"
    // 2. (+ 1) advances pointer on the new string
    char *offset = (h @ w) + 1;
    // Result: "elloWorld"

    return 0;
}
```

A.4.4 Nested Expressions and Escaping

Based on the implementation, F-strings can be nested arbitrarily deep, enabling complex formatting logic.

```c
int main() {
    int val = 100;

    // Nested F-String:
    // The inner expression { f"[{val}]" } evaluates to "[100]" first.
    char *debug = f"Debug Info: { f"Value -> [{val}]" }";

    return 0;
}
```

A.4.5 Compiler Error Handling

The compiler enforces strict type safety for the `@` operator to prevent accidental pointer miscalculations.

```c
int main() {
    char *base = "Error Code: ";
    int code = 404;

    // INVALID: Cannot concatenate string (char *) with integer (int)
    // char *err = base @ code;  <-- Compilation Error

    // CORRECT: Use F-String to convert the integer first
    char *msg = base @ f"{code}";
    // Result: "Error Code: 404"

    return 0;
}
```

B Appendix: Base YAPL Language Specification

This appendix details the specification for the base YAPL language (a subset of C11) as available here: YAPL Language Specification (PDF).

B.1 Language Constructs

The language conforms to the ANSI C-2011 standard with the following specific inclusions:

- global variables, both normal as well as `extern`.
- function declaration, function definition, and function call (including recursion).
- `for` loop, `while` loop, `do-while`.
- `if`, `if-else`, `if-else-if` (nested/ladder) (if condition can be any expression including literals/constants).
- `switch/case` with case labeled with `:`; must support fall-through; must support the optional `default` clause.
- variable declaration, definition, initialization locally/globally (multiple variables comma-separated).
- strings (of the forms `"..."` (single), `"..." "..."` (multiple)), character, integer and floating point literals.
- single-line (`//`) and multi-line (`/*...*/`) comments (multi-line comments should be properly closed lexically).
- `break`, `continue`, and `return` statements.
- a statement must end in a semi-colon `;` like the usual standard of C.

B.2 Supported Data Types

- **Primary:** `int`, `long`, `short`, `float`, `double`, `void`, `char` and their pointers.
- **User-defined:** structures (`struct`) (pointer dereferences must support arbitrary level of indirection).
- **Derived:** functions, arrays, and pointers (all supported data types; no need to support function pointers).

B.3 Supported Operators

Operator precedence and associativity follow standard C rules.

- **Relational operators:** `<`, `>`, `==`, `<=`, `>=`, `!=`, `<=>`.
- **Unary operators:** `+`, `-`, `!`, `*`, `&`, `~`.
- **Binary operators:** `+`, `-`, `*`, `&`, `|`, `/`, `%`, `^`.
- **Logical operators:** `&&`, `||`.
- **Assignment operator:** `=`.
- **Suffix/postfix increment and decrement:** `++`, `--`.

- **Structure field access operators:** `->`, `.`.
- **Pointers:** `*`, `&` (pointer dereferences must support arbitrary/multiple level of indirection, i.e., `int *****p;`).
- **Function call:** `()` (any valid expression inside including blank, constants, and literals).
- **Array subscripting:** `[]` (any valid expression inside including constants and literals).
- `sizeof()`**:** operands: `<typename>` or unary expression.
- **Typecast:** `(<typename>)`.

B.4 Unsupported Constructs

The following C11 features are **explicitly excluded**:

- **Preprocessors:** none allowed (no `#include`s, `#define`s needed anywhere).
- **Operators:** `...` (ellipsis), `?:` (ternary operator) (function vararg list specification not required).
- **Operators:** `>>=`, `<<=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `^=`, `|=`, `<<`, `>>`.
- **Constructs/keywords:** `auto`, `const`, `goto`, `inline`, `register`, `restrict`, `signed`, `static`, `typedef`, `unsigned`, `enum`, `union`, `volatile`, `_Alignas`, `_Alignof`, `_Atomic`, `_Bool`, `_Complex`, `_Generic`, `_Imaginary`, `_Noreturn`, `_Static_assert`, `_Thread_local`, `__func__`.